

Atelier 13 : **Javascript**

SOMMAIRE

Atelier 13 : Javascript 2/2 : Navigateur 3.....	3
Exercice.....	3
Cacher sur clic.....	3
Se cacher.....	4
Quels handlers exécutent?.....	5
Déplacez le ballon à travers le terrain.....	6
Créez un menu glissant.....	8
Ajoutez un bouton de fermeture.....	11
Carrousel.....	14
Atelier 13 : Javascript 2/2 : Navigateur 4.....	17
Liste sélectionnable.....	17
Info-bulle ‘intelligente’.....	20
Raccourcis clavier étendus.....	22

Atelier 13 : Javascript 2/2 : Navigateur 3

Exercice

Cacher sur clic

Cacher sur clic

Ajouter JavaScript à la `button` pour faire `<div id="text">` disparaître quand on clique dessus.

Dans cet exercice nous allons utiliser une fonction pour faire disparaître un élément `<div>` lorsque l'utilisateur clique sur un bouton

Nous commençons par sélectionner le bouton et la div à l'aide de leurs identifiants. Ensuite, nous ajoutons un gestionnaire d'événement click sur le bouton grâce à `addEventListener`. Lorsque l'utilisateur clique sur le bouton, la fonction associée est exécutée et modifie la propriété `display` de la div pour la rendre invisible. Cet exercice permet de comprendre comment réagir à une action de l'utilisateur avec JavaScript.

```
<script>
  let button = document.getElementById("btn");
  let text = document.getElementById("text");

  button.addEventListener("click", function () {
    text.style.display = "none";
  });
</script>
```

Clique ici



Ce texte va disparaître quand on clique sur le bouton.

Clique ici

Se cacher

Se cacher

Crée un bouton qui se cache tout seul au clic.

Nous allons créer un bouton qui se cache tout seul lorsqu'on clique dessus. Nous allons utiliser JavaScript pour détecter le clic et modifier le style du bouton afin de le rendre invisible.

```
<script>
  let button = document.getElementById("btn");

  button.addEventListener("click", function () {
    button.style.display = "none";
  });
</script>
```

Clique pour me cacher

Quels handlers exécutent?

Quels handlers exécutent?

Il y a un bouton dans la variable. Il n'y a aucun handlers dessus.

Quels handlers s'exécutent au clic après le code suivant? Quelles alertes apparaissent?

```
1 button.addEventListener("click", () => alert("1"));
2
3 button.removeEventListener("click", () => alert("1"));
4
5 button.onclick = () => alert(2);
```

Nous allons analyser quels gestionnaires d'événements sont réellement exécutés lorsqu'on clique sur un bouton, en tenant compte de l'ajout et de la suppression d'événements ainsi que de l'utilisation de la propriété onclick.

Les deux gestionnaires s'exécutent. L'alerte « 1 » s'affiche car `removeEventListener` ne supprime pas le handler ajouté, puis l'alerte « 2 » s'affiche via la propriété onclick.

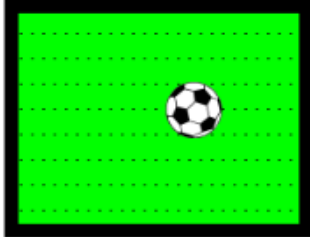
Déplacez le ballon à travers le terrain

Déplacez le ballon à travers le terrain

Déplacez le ballon à travers le terrain jusqu'à un clic.

[ballon2.html](#)

Click on a field to move the ball there.



Exigences:

- Le centre de la balle devrait passer exactement sous le pointeur au clic (si possible sans franchir le bord du champ).
- L'animation CSS est la bienvenue.
- La balle ne doit pas franchir les limites du terrain.
- Quand la page est défilée, rien ne devrait se casser.

Notes:

- Le code doit aussi fonctionner avec différentes tailles de balle et de champ, sans être lié à des valeurs fixes.
- Propriétés d'utilisation `event.clientX/event.clientY` pour les coordonnées de clic.

Dans cet exercice, nous allons déplacer la balle vers l'endroit où l'utilisateur clique dans le terrain. Le centre de la balle devra passer exactement sous le pointeur de la souris, sans jamais sortir du terrain, même si la taille du champ ou de la balle change. Le déplacement sera calculé à partir des coordonnées du clic et des dimensions réelles des éléments du DOM.

Nous commençons par attendre le chargement de l'image de la balle afin de connaître ses dimensions réelles. Lors d'un clic dans le terrain, nous récupérons les coordonnées de la souris avec `event.clientX` et `event.clientY`, puis nous les convertissons en coordonnées relatives au terrain grâce à `getBoundingClientRect`. Nous ajustons ensuite ces coordonnées pour que le centre de la balle corresponde exactement au point cliqué. Enfin, nous limitons la position calculée pour empêcher la balle de sortir du terrain, puis nous appliquons les nouvelles positions à la balle.

```
<script>
  let field = document.getElementById("field");
  let ball = document.getElementById("ball");

  ball.onload = function () {

    field.addEventListener("click", function (event) {

      let fieldRect = field.getBoundingClientRect();

      // Position du clic dans le terrain
      let x = event.clientX - fieldRect.left;
      let y = event.clientY - fieldRect.top;

      // Centrer la balle sur le clic
      x -= ball.offsetWidth / 2;
      y -= ball.offsetHeight / 2;

      // Empêcher la balle de sortir du terrain
      x = Math.max(0, Math.min(x, field.clientWidth - ball.offsetWidth));
      y = Math.max(0, Math.min(y, field.clientHeight - ball.offsetHeight));

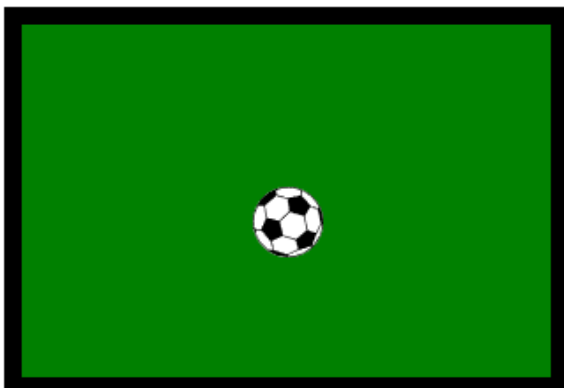
      ball.style.left = x + "px";
      ball.style.top = y + "px";

    });

  };
</script>
```

Résultat

Click on a field to move the ball there.



Créez un menu glissant

Créez un menu glissant

Crée un menu qui s'ouvre/replie au clic



P.S. HTML/CSS du document source doit être modifié. [menu.html](#)

Dans cet exercice, nous allons créer un menu glissant qui s'ouvre et se replie lorsque l'on clique sur son titre. Nous allons modifier le HTML, le CSS pour l'animation, et utiliser JavaScript pour gérer le clic et basculer l'affichage du menu.

En CSS, nous masquons la liste grâce à `max-height: 0` et nous ajoutons une transition pour créer l'effet de glissement.

```
<style>
  .menu {
    width: 200px;
    border: 1px solid #ccc;
    padding: 5px;
    font-family: Arial, sans-serif;
  }

  .menu-title {
    cursor: pointer;
    user-select: none;
    font-weight: bold;
  }

  .menu-title::before {
    content: "► ";
  }

  .menu.open .menu-title::before {
    content: "▼ ";
  }

  .menu-list {
    margin-left: 15px;
    max-height: 0;
    overflow: hidden;
    transition: max-height 0.3s ease;
  }

  .menu.open .menu-list {
    max-height: 100px;
  }
</style>
```

En JavaScript, nous écoutons le clic sur le titre du menu et nous ajoutons ou retirons une classe CSS. Cette classe permet d'afficher ou de cacher la liste et de changer l'icône du menu.

```
<script>
  let menu = document.getElementById("menu");
  let title = menu.querySelector(".menu-title");

  title.addEventListener("click", function () {
    menu.classList.toggle("open");
  });
</script>
```

▶ **Sweeties (click me!)**



▼ **Sweeties (click me!)**

Cake
Donut
Honey

Nous avons créé un menu glissant qui s'ouvre et se replie au clic.

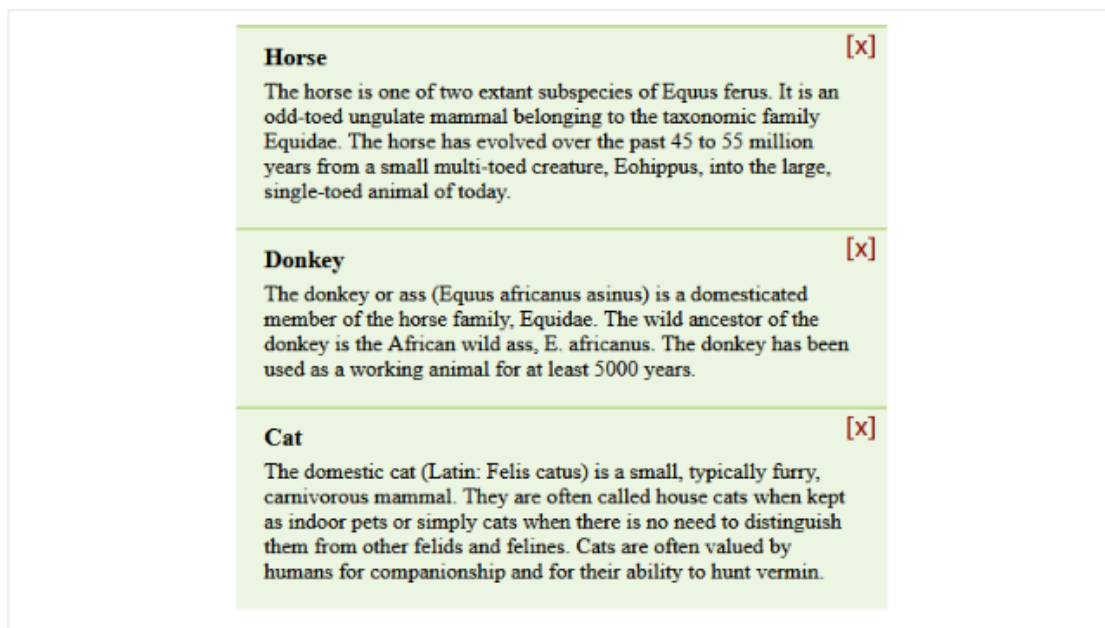
Ajoutez un bouton de fermeture

Ajoutez un bouton de fermeture

Il y a une liste de messages.

Utilisez JavaScript pour ajouter un bouton de fermeture dans le coin supérieur droit de chaque message.

Le résultat devrait ressembler à ceci:



[fermeture.html](#)

[fermeture.css](#)

Nous allons ajouter dynamiquement un bouton de fermeture [x] dans le coin supérieur droit de chaque message. Lorsqu'on clique sur ce bouton, le message correspondant disparaît. Nous allons utiliser JS pour créer le bouton et gérer le clic, et le CSS fourni pour le positionnement et le style.

Nous avons créé plusieurs blocs de messages et ajouté leur style directement dans la balise `<style>`. Ensuite, en Javascript, nous avons parcouru chaque message pour y ajouter dynamiquement un bouton de fermeture. Lorsqu'un utilisateur clique sur ce bouton, nous supprimons uniquement le message correspondant. Le positionnement du bouton est assuré grâce au CSS.

```
<div class="pane">
  <h3>Horse</h3>
  <p>
    The horse is one of two extant subspecies of Equus ferus. It is an odd-toed ungulate mammal belonging
  </p>
</div>

<div class="pane">
  <h3>Donkey</h3>
  <p>
    The donkey or ass (Equus africanus asinus) is a domesticated member of the horse family, Equidae. The
  </p>
</div>

<div class="pane">
  <h3>Cat</h3>
  <p>
    The domestic cat (Latin: Felis catus) is a small, typically furry, carnivorous mammal. They are often
  </p>
</div>

<script>
  let panes = document.querySelectorAll(".pane");

  panes.forEach(pane => {
    let button = document.createElement("button");
    button.className = "remove-button";
    button.textContent = "[x]";

    button.addEventListener("click", () => {
      pane.remove();
    });

    pane.append(button);
  });
</script>
```

Horse



The horse is one of two extant subspecies of *Equus ferus*. It is an odd-toed ungulate mammal belonging to the taxonomic family Equidae. The horse has evolved over the past 45 to 55 million years from a small multi-toed creature, *Eohippus*, into the large, single-toed animal of today.

Donkey



The donkey or ass (*Equus africanus asinus*) is a domesticated member of the horse family, Equidae. The wild ancestor of the donkey is the African wild ass, *E. africanus*. The donkey has been used as a working animal for at least 5000 years.

Cat

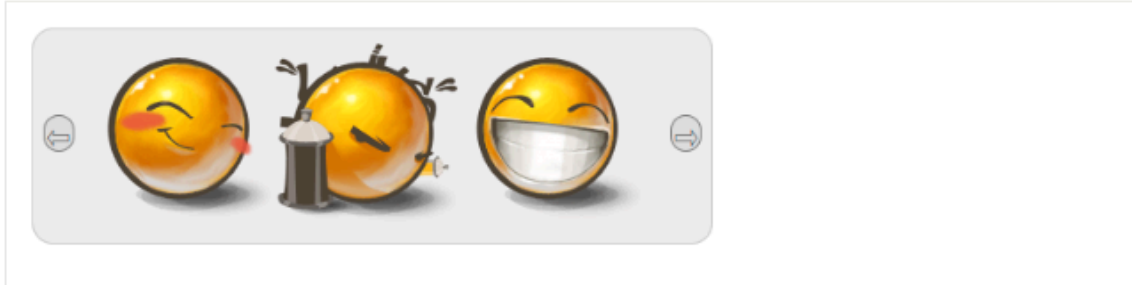


The domestic cat (Latin: *Felis catus*) is a small, typically furry, carnivorous mammal. They are often called house cats when kept as indoor pets or simply cats when there is no need to distinguish them from other felids and felines. Cats are often valued by humans for companionship and for their ability to hunt vermin.

Carrousel

Carrousel

Créez un « carrousel » – un ruban d’images que l’on peut faire défiler en cliquant sur des flèches.



P.S. Pour cette tâche, la structure HTML/CSS représente en fait 90% de la solution.

[carrousel.html](#)

[carrousel.css](#)

Dans cet exercice, nous allons créer un carrousel d’images à partir de la structure HTML et du CSS fournis. Les images sont disposées dans un ruban horizontal, et nous allons utiliser uniquement JavaScript pour déplacer ce ruban vers la gauche ou vers la droite lorsque l’utilisateur clique sur les flèches, tout en respectant les limites du carrousel.

Les images sont placées dans une liste très large, et seule une partie est visible grâce à la propriété `overflow: hidden`. En JS, nous modifions la marge gauche de la liste lorsque l'utilisateur clique sur les flèches afin de faire défiler les images. Nous limitons le déplacement pour empêcher le carrousel de dépasser ses bornes.

```
<style>
.arrow {
  padding: 0;
  background: #ddd;
  border-radius: 15px;
  border: 1px solid #gray;
  font-size: 24px;
  line-height: 24px;
  color: #444;
  display: block;
}

.arrow:focus {
  outline: none;
}

.arrow:hover {
  background: #ccc;
  cursor: pointer;
}

ul {
  height: 130px;
  width: 9999px;
  margin: 0;
  padding: 0;
  list-style: none;
  font-size: 0;
}

ul img {
  width: 130px;
  height: 130px;
  display: block;
}

ul li {
  display: inline-block;
}

.carousel {
  width: 390px;
  padding: 10px;
  border-radius: 10px;
  background: #eee;
}
```

```
<div class="carousel">
  <button class="arrow prev"></button>

  <div class="gallery">
    <ul>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
    </ul>
  </div>

  <button class="arrow next"></button>
</div>

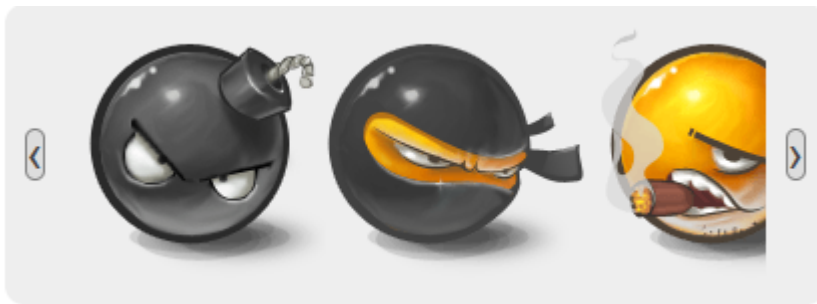
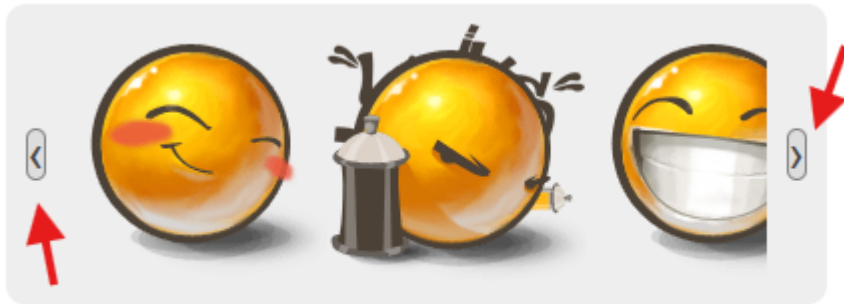
<script>
  let position = 0;
  let imageWidth = 130;
  let visibleImages = 3;

  let ul = document.querySelector("ul");
  let imagesCount = ul.querySelectorAll("li").length;

  document.querySelector(".prev").onclick = function () {
    position += imageWidth;
    position = Math.min(position, 0);
    ul.style.marginLeft = position + "px";
  };

  document.querySelector(".next").onclick = function () {
    position -= imageWidth;
    let maxPosition = -imageWidth * (imagesCount - visibleImages);
    position = Math.max(position, maxPosition);
    ul.style.marginLeft = position + "px";
  };
</script>
```

Résultat



Atelier 13 : Javascript 2/2 : Navigateur 4

Liste sélectionnable

Liste sélectionnable

Créer une liste dont les éléments sont sélectionnables, comme dans les gestionnaire de fichiers

- Un click sur un élément de la liste sélectionne seulement cet élément (ajoute la classe `.selected`), désélectionne tous les autres.
- Si un click est effectué avec `Ctrl` (`Cmd` sur Mac), alors la sélection est inversée sur l'élément, mais les autres éléments ne sont pas modifiés

Cliquez sur un élément de la liste pour le sélectionner.

- Christopher Robin
- Winnie-the-Pooh
- Tigger
- Kanga
- Rabbit. Just rabbit.

P.S. Pour cette tâche on peut assumer que les éléments de cette liste sont uniquement du texte. Pas de tags imbriqués.

P.P.S. Empêcher la sélection des textes déclenchée par défaut par le navigateur lors d'un click.

[liste.html](#)

Nous allons créer une liste sélectionnable, comme dans un gestionnaire de fichiers.

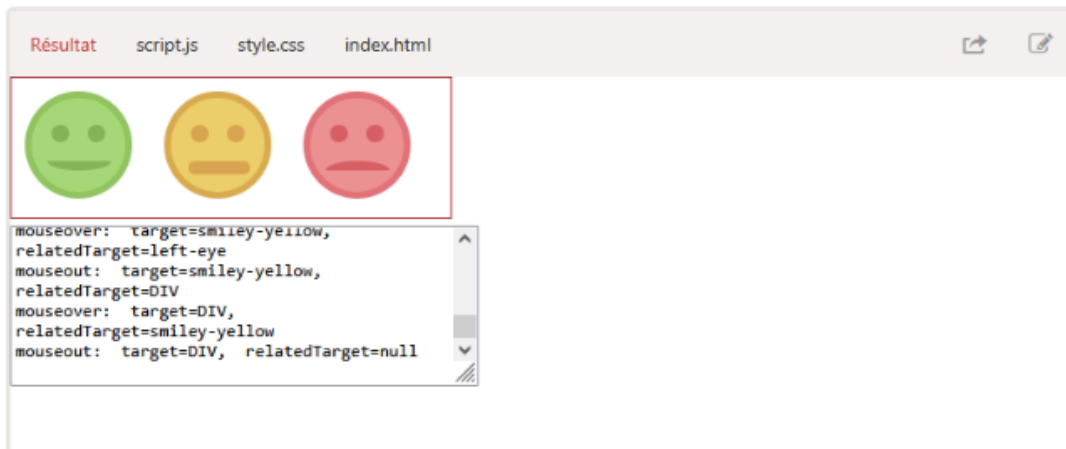
Un clic simple sélectionne uniquement l'élément cliqué, tandis qu'un clic avec la touche Ctrl permet d'inverser la sélection de l'élément sans modifier les autres. Nous allons également empêcher la sélection du texte par défaut du navigateur.

Cliquez sur un élément de la liste pour le sélectionner.

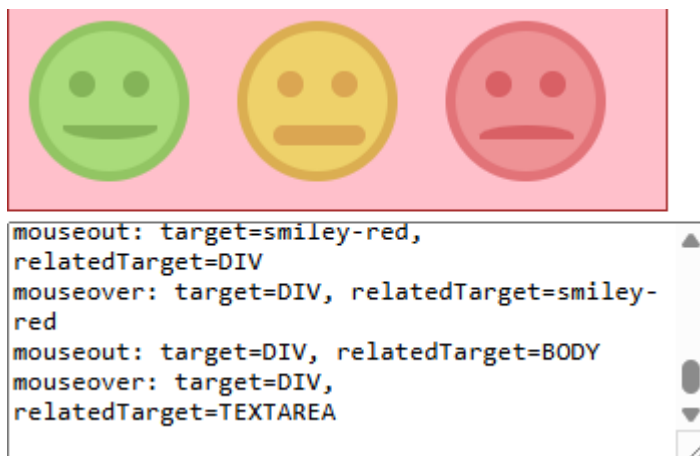
- Christopher Robin
- Winnie-the-Pooh
- Tigger
- Kanga
- Rabbit. Just rabbit.

Dans l'exemple ci-dessous chaque aspect facial est un élément. Lorsque vous déplacez la souris, vous pouvez voir les événements de souris dans la zone de texte.

Chaque événement contient les informations sur `target` et `relatedTarget` :



[exemple.zip](#)



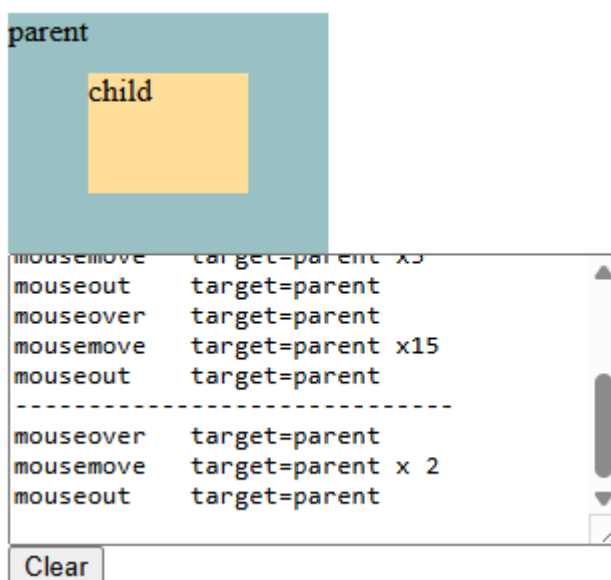
Vous pouvez le vérifier "en direct" sur un banc d'essai ci-dessous.

Son code HTML comporte deux éléments imbriqués: la `<div id="child">` est à l'intérieur de `<div id="parent">`. Si vous déplacez rapidement la souris dessus, alors peut-être que seule la div enfant déclenchera les événements, ou peut-être la div parent, ou peut-être qu'il n'y aura aucun événement.

Déplacez également le pointeur dans la `div` enfant, puis le déplacer rapidement en dehors à travers le parent. Si le mouvement est assez rapide, l'élément parent est ignoré. La souris traversera l'élément parent sans le remarquer.



[banc.zip](#)



Info-bulle ‘intelligente’

Info-bulle “Intelligente”

Écrire JavaScript qui montre une infobulle sur un élément avec l'attribut data-tooltip. La valeur de cet attribut devrait devenir le texte de l'infobulle.

C'est comme la tâche Comportement info-bulle, mais ici les éléments annotés peuvent être imbriqués. L'infobulle la plus profondément imbriquée est montrée.

Une seule infobulle peut apparaître en même temps.

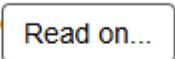
Exemple :

```
<div data-tooltip="Here - is the house interior" id="house">
  <div data-tooltip="Here - is the roof" id="roof"></div>
  ...
  <a href="https://en.wikipedia.org/wiki/The_Three_Little_Pigs" data-tooltip="Read on...">Hover over me</a>
</div>
```

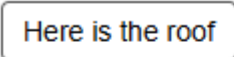
Dans cet exercice, nous allons créer une info-bulle intelligente en JavaScript. Lorsqu'on survole un élément qui possède l'attribut data-tooltip, une info-bulle apparaît avec le texte contenu dans cet attribut. Si des éléments avec data-tooltip sont imbriqués, seule l'info-bulle la plus profonde est affichée. Une seule info-bulle peut être visible à la fois.

Il était une fois, une maman cochon qui avait trois petits cochons.

Bientôt ils rencontrèrent un homme. [Survolez moi](#)

Il était une fois, une maman cochon  s petits cochons.

Bientôt ils rencontrèrent un homme. [Survolez moi](#)

 Here is the roof

Il était une fois, une maman cochon qui avait trois petits cochons.

Bientôt ils rencontrèrent un homme. [Survolez moi](#)

```
<script>
  let tooltip;

  document.addEventListener("mouseover", function (event) {
    let target = event.target.closest("[data-tooltip]");
    if (!target) return;

    tooltip = document.createElement("div");
    tooltip.className = "tooltip";
    tooltip.textContent = target.dataset.tooltip;

    document.body.append(tooltip);

    let coords = target.getBoundingClientRect();
    tooltip.style.left = coords.left + "px";
    tooltip.style.top = coords.top - tooltip.offsetHeight - 5 + "px";
  });

  document.addEventListener("mouseout", function (event) {
    if (tooltip) {
      tooltip.remove();
      tooltip = null;
    }
  });
</script>
```

Raccourcis clavier étendus

Raccourcis clavier étendus

Créer une fonction `runOnKeys(func, code1, code2, ... code_n)` exécutant la fonction `func` lorsqu'on appuie simultanément sur les touches avec les codes suivant `code1, code2, ..., code_n`.

Par exemple, le code ci-dessous montre `alert` lorsque "Q" et "W" sont appuyées ensemble (dans n'importe quelle langue, avec ou sans l'activation de La touche Majuscule, CapsLock)

```
1 runOnKeys(  
2   () => alert("Hello!"),  
3   "KeyQ",  
4   "KeyW"  
5 );
```

Aide :

Nous devons utiliser deux gestionnaires: `document.onkeydown` et `document.onkeyup`.

L'ensemble `pressed` doit garder les touches en cours appuyées.

Créons un set `pressed = new Set()` pour garder les touches actuellement enfoncées.

Le premier gestionnaire en ajoute, tandis que le second en supprime. Chaque fois sur `keydown` nous vérifions si nous avons suffisamment de touches enfoncées et exécutons la fonction si c'est le cas.

Dans cet exercice, nous allons créer une fonction `runOnKeys` qui exécute une fonction donnée lorsque plusieurs touches du clavier sont appuyées simultanément.

La détection doit fonctionner quelle que soit la langue du clavier et sans tenir compte de Majuscule. Nous allons utiliser les événements `keydown` et `keyup` ainsi qu'un `Set` pour mémoriser les touches actuellement enfoncées.

Nous allons utiliser un `Set` pour mémoriser les touches actuellement enfoncées. Lors d'un événement `keydown`, nous ajoutons la touche dans le `Set` et vérifions si toutes les touches demandées sont présentes. Si c'est le cas, nous exécutons la fonction fournie. Lors d'un événement `keyup`, nous supprimons la touche relâchée du `Set`, ce qui permet de garder une liste à jour des touches enfoncées.

```
<p>Appuyez sur Q et W en même temps.</p>

<script>
  function runOnKeys(func, ...codes) {
    let pressed = new Set();

    document.addEventListener("keydown", function (event) {
      pressed.add(event.code);

      for (let code of codes) {
        if (!pressed.has(code)) {
          return;
        }
      }

      func();
      pressed.clear();
    });

    document.addEventListener("keyup", function (event) {
      pressed.delete(event.code);
    });
  }

  runOnKeys(
    () => alert("Hello!"),
    "KeyQ",
    "KeyW"
  );
</script>
```

Appuyez sur Q et W en même temps.

Cette page indique

Hello!

OK